

Отчет к лабораторной работе №3. ИГИ.
Python. Стандартные типы данных, коллекции, функции, модули.

Студента гр. 453504
Романова Николая

Вариант 3

Задание 1. В соответствии с заданием своего варианта составить программу для вычисления значения функции с помощью разложения функции в степенной ряд. Задать точность вычислений ϵ .

Предусмотреть максимальное количество итераций, равное 500.

Вывести количество членов ряда, необходимых для достижения указанной точности вычислений. Результат получить в виде:

x	n	$F(x)$	$Math F(x)$	ϵ

Здесь x – значение аргумента, $F(x)$ – значение функции, n – количество просуммированных членов ряда, $Math F(x)$ – значение функции, вычисленное с помощью модуля `math`.

$$\ln(1+x) = \sum_{n=0}^{\infty} (-1)^{n-1} \frac{x^n}{n} = x - \frac{x^2}{2} + \frac{x^3}{3} + \dots, |x| < 1$$

Код задания 1:

```
'''
'''

Purpose: Task 1 - Calculating function value using power series
expansion

Lab number: 3

Lab title: Standard data types, collections, functions, modules

Version: 1.0
```

Developer: Romanov Nikolai Viktorovich

Date: 08.03.2026

"""

```
import math
```

```
from validation import inputEpsila, inputArgumentX
```

```
def calculate_exact_f(x):
```

"""

Calculates the exact value of the function using the math module.

Formula: $\ln(1 + x)$

"""

```
    return math.log(1 + x)
```

```
def taylor_series_term(n, x):
```

"""

Calculates the n-th term of the Taylor series for $\ln(1 + x)$.

Formula: $(-1)^{(n-1)} * (x^n) / n$

"""

```
    return ((-1) ** (n - 1)) * (x ** n) / n
```

```
def print_result_table(x, n, f_value, exact_value, eps):
```

"""

Prints the calculated data in a formatted table matching the task requirements.

```

"""

print('\n' + '-' * 75)

print(f'| {"x":^10} | {"n":^5} | {"F(x)":^15} | {"Math F(x)":^15} | {"eps":^10} |')

print('-' * 75)

print(f'| {x:^10.4f} | {n:^5} | {f_value:^15.6f} | {exact_value:^15.6f} | {eps:^10.6f} |')

print('-' * 75 + '\n')

```

```
def task1():
```

```

"""

Main business function for Task 1.

Handles user input, performs the iterative calculation of the power
series,

and calls the output function.

"""

print("--- Task 1: Power Series Expansion ---")

eps = inputEpsila("Enter epsilon (calculation accuracy): ")

x = inputArgumentX("Enter argument x (|x| < 1): ")

max_iter = 500

exact_value = calculate_exact_f(x)

f_value = 0.0

n = 0

for i in range(1, max_iter + 1):

```

```

term = taylor_series_term(i, x)

f_value += term

if abs(term) <= eps:

    n = i

    break

else:

    n = max_iter

    print(f"\nWarning: Maximum iterations ({max_iter}) reached
without achieving required accuracy.")

print_result_table(x, n, f_value, exact_value, eps)

```

Демонстрация задания 1:

```

=====

--- Task 1: Power Series Expansion ---
Enter epsilon (calculation accuracy): 0.000001
Enter argument x (|x| < 1): 0.1

-----
|      x      |  n  |      F(x)      |  Math F(x)  |  eps  |
-----
|  0.1000  |  6  |  0.095310  |  0.095310  |  0.000001  |
-----

Options:
1) Repeat the task
2) Exit to main menu / Quit
Enter your choice (1 or 2): █

```

Задание 2. В соответствии с заданием своего варианта составить программу для нахождения суммы последовательности чисел.

Организовать цикл, принимающий целые числа и подсчитывающий количество положительных. Окончание – ввод 10

Код задания 2:

```
"""
Purpose: Task 2 - Counting positive numbers in a sequence until 10 is
entered
Lab number: 3
Lab title: Standard data types, collections, functions, modules
Version: 1.0
Developer: Romanov Nikolai Viktorovich
Date: 08.03.2026
"""

from validation import inputIntNum

def print_start_message():
    """
    Prints the introductory message and instructions for Task 2.
    """
    print("\n--- Task 2: Count positive numbers ---")
    print("Enter integers one by one.")
    print("To finish the input and calculate the result, enter the
number 10.\n")

def print_end_message(count):
    """
    Prints the final count of positive numbers.
    """
    print("-" * 45)
    print(f"Total count of positive numbers entered: {count}")
    print("-" * 45 + "\n")

def task2():
    """
    Main business function for Task 2.
    Loops to accept integers from the user, counts positive ones,
    and stops when 10 is entered.
    """
    print_start_message()
    pos_count = 0

    while True:
```

```

num = inputIntNum("Enter an integer: ")

if num == 10:
    break
elif num > 0:
    pos_count += 1

print_end_message(pos_count)

```

Демонстрация задания 2:

```

--- Task 2: Count positive numbers ---
Enter integers one by one.
To finish the input and calculate the result, enter the number 10.

Enter an integer: 1
Enter an integer: 2
Enter an integer: 4
Enter an integer: 8
Enter an integer: 10
-----
Total count of positive numbers entered: 4
-----

Options:
1) Repeat the task
2) Exit to main menu / Quit
Enter your choice (1 or 2): █

```

Задание 3. Не использовать регулярные выражения. В соответствии с заданием своего варианта составить программу для анализа текста, вводимого с клавиатуры.

Определить, является ли введенная с клавиатуры строка шестнадцатеричным числом

Код задания 3:

```

"""

Purpose: Task 3 - String analysis (Check if string is a hexadecimal
number)

Lab number: 3

```

Lab title: Standard data types, collections, functions, modules

Version: 1.0

Developer: Romanov Nikolai Viktorovich

Date: 08.03.2026

"""

```
from validation import inputStr
```

```
def is_hex_number(text):
```

```
    """
```

```
    Analyzes the string to check if it represents a hexadecimal number.
```

```
    Does NOT use regular expressions. Handles optional '0x' prefix and signs.
```

```
    """
```

```
    clean_text = text.strip().lower()
```

```
    if clean_text.startswith('-') or clean_text.startswith('+):
```

```
        clean_text = clean_text[1:]
```

```
    if clean_text.startswith('0x'):
```

```
        clean_text = clean_text[2:]
```

```
    if not clean_text:
```

```
        return False
```

```

valid_hex_chars = "0123456789abcdef"

for char in clean_text:

    if char not in valid_hex_chars:

        return False

return True


def print_hex_result(is_hex, original_text):

    """

    Prints the result of the hexadecimal check in a friendly format.

    """

    print("\n" + "-" * 45)

    if is_hex:

        print(f'SUCCESS: "{original_text}" IS a valid hexadecimal
number.')

    else:

        print(f'ERROR: "{original_text}" IS NOT a valid hexadecimal
number.')

    print("-" * 45 + "\n")


def task3():

    """

    Main business function for Task 3.

    Accepts a string from the user, checks if it's hex, and prints the
result.

    """

```



```
print("\n--- Task 3: Hexadecimal Number Analyzer ---")

user_string = inputStr("Enter a string to check: ")

result = is_hex_number(user_string)

print_hex_result(result, user_string)
```

Демонстрация задания 3:

```
--- Task 3: Hexadecimal Number Analyzer ---
Enter a string to check: 15fa

-----
SUCCESS: "15fa" IS a valid hexadecimal number.
-----

Options:
1) Repeat the task
2) Exit to main menu / Quit
Enter your choice (1 or 2): █
```

Задание 4. Не использовать регулярные выражения. Дана строка текста, в которой слова разделены пробелами и запятыми. В соответствии с заданием своего варианта составьте программу для анализа строки, инициализированной в коде программы:

«So she was considering in her own mind, as well as she could, for the hot day made her feel very sleepy and stupid, whether the pleasure of making a daisy-chain would be worth the trouble of getting up and picking the daisies, when suddenly a White Rabbit with pink eyes ran close by her.»

Если не оговорено иное, то регистр букв при решении задачи не имеет значения.

а) определить количество слов в строке и вывести на экран все слова, количество букв у которых четное;

б) найти самое короткое слово, которое начинается на 'a';

в) вывести повторяющиеся слова

Код задания 4:

```
"""
Purpose: Task 4 - String analysis without regular expressions
Lab number: 3
Lab title: Standard data types, collections, functions, modules
Version: 1.0
Developer: Romanov Nikolai Viktorovich
Date: 08.03.2026
"""

def get_cleaned_words(text):
    """
    Splits text into words and removes punctuation marks (, .)
    from the edges of each word without using regex.
    """
    raw_words = text.split()
    cleaned_words = []

    for word in raw_words:
        clean_word = word.strip('.,')

        if clean_word:
            cleaned_words.append(clean_word)
```

```

    return cleaned_words

def get_even_length_words(words):
    """
    Returns a list of words that have an even number of characters.
    """
    return[word for word in words if len(word) % 2 == 0]

def find_shortest_word(words, start_char='a'):
    """
    Finds the shortest word starting with a specific character
    (case-insensitive).
    """
    matching_words =[word for word in words if
word.lower().startswith(start_char.lower())]

    if not matching_words:

        return None

    return min(matching_words, key=len)

def get_repeating_words(words):
    """
    Returns a list of words that appear more than once in the text
    (case-insensitive).
    """

```

```

lower_words = [word.lower() for word in words]

repeats = set()

seen = set()

for word in lower_words:

    if word in seen:

        repeats.add(word)

    else:

        seen.add(word)

return list(repeats)

def print_task4_results(total_words, even_words, shortest_a_word,
repeat_words):

    """

    Prints the results of Task 4 in a formatted and user-friendly way.

    """

    print("\n" + "=" * 60)

    print(f"{'TASK 4 RESULTS':^60}")

    print("=" * 60)

    print(f"a) Total words in text: {total_words}")

    print(f"    Words with even length ({len(even_words)}):")

    print(f"    {'', '.join(even_words)}\n")

```

```

print(f"b) Shortest word starting with 'a':")

if shortest_a_word:

    print(f"    -> '{shortest_a_word}'\n")

else:

    print("    -> No such words found.\n")


print(f"c) Repeating words ({len(repeat_words)}):")

print(f"    {' '.join(repeat_words)}")

print("=" * 60 + "\n")


def task4():

    """

    Main business function for Task 4.

    Initializes the hardcoded string and orchestrates the sub-tasks.

    """

    print("\n--- Task 4: Text Analysis ---")


    text = (

        "So she was considering in her own mind, as well as she could, "

        "for the hot day made her feel very sleepy and stupid, whether "

        "the pleasure of making a daisy-chain would be worth the trouble

"

        "of getting up and picking the daisies, when suddenly a White

Rabbit "

        "with pink eyes ran close by her."

    )

```

```

print(f"Analyzing text:\n\"{text}\"\\n")

words = get_cleaned_words(text)

total_count = len(words)

even_words = get_even_length_words(words)

shortest_a = find_shortest_word(words, 'a')

repeating = get_repeating_words(words)

print_task4_results(total_count, even_words, shortest_a, repeating)

```

Демонстрация задания 4:

--- Task 4: Text Analysis ---

Analyzing text:

"So she was considering in her own mind, as well as she could, for the hot day made her feel very sleepy and stupid, whether the pleasure of making a daisy-chain would be worth the trouble of getting up and picking the daisies, when suddenly a White Rabbit with pink eyes ran close by her."

TASK 4 RESULTS

a) Total words in text: 55

Words with even length (24):

So, in, mind, as, well, as, made, feel, very, sleepy, stupid, pleasure, of, making, be, of, up, when, suddenly, Rabbit, with, pink, eyes, by

b) Shortest word starting with 'a':

-> 'a'

c) Repeating words (7):

as, and, her, she, the, of, a

Options:

1) Repeat the task

2) Exit to main menu / Quit

Enter your choice (1 or 2): █

Задание 5. В соответствии с заданием своего варианта составить программу для обработки вещественных списков. Программа должна содержать следующие базовые функции:

- 1) ввод элементов списка пользователем;
- 2) проверка корректности вводимых данных;
- 3) реализация основного задания с выводом результатов;
- 4) вывод списка на экран.

Найти максимальный по модулю элемент списка и сумму элементов списка расположенных до последнего положительного элемента.

Код задания 5:

```
"""
Purpose: Task 5 - Processing lists of float numbers
Lab number: 3
Lab title: Standard data types, collections, functions, modules
Version: 1.0
Developer: Romanov Nikolai Viktorovich
Date: 08.03.2026
"""

from validation import inputSizeList, inputIntNum
import init_list

def find_max_abs_element(lst):
    """
    Finds and returns the element with the maximum absolute value.
    """
    if not lst:
        return 0.0
```

```

    # Pythonic way: ищем максимум, используя abs() как критерий
    сравнения

    return max(lst, key=abs)

def sum_before_last_positive(lst):
    """
    Calculates the sum of elements located before the last positive
    element.
    """
    last_pos_index = -1

    for i in range(len(lst) - 1, -1, -1):
        if lst[i] > 0:
            last_pos_index = i
            break

    if last_pos_index == -1:
        return None

    return sum(lst[:last_pos_index])

def print_task5_results(lst, max_abs, sum_before):
    """
    Prints the list and the calculation results (Lab Task 5 req 3 and
    4).
    """
    print("\n" + "=" * 50)
    print(f"{'TASK 5 RESULTS':^50}")
    print("=" * 50)

```



```

print("1) Original List:")

print(f"    {lst}\n")

print("2) Element with maximum absolute value:")

print(f"    {max_abs}\n")

print("3) Sum of elements before the last positive one:")

if sum_before is not None:

    print(f"    {round(sum_before, 2)}")

else:

    print("    -> No positive elements found in the list.")

print("=" * 50 + "\n")

def task5():

    """

    Main business function for Task 5.

    """

    print("\n--- Task 5: Float List Processing ---")

    size = inputSizeList("Enter the size of the list (integer > 0): ")

    print("\nChoose initialization method:")

    print("1) Manual input")

    print("2) Random generation (using generator)")

    while True:

        choice = inputIntNum("Enter your choice (1 or 2): ")

        if choice == 1:

            sequence = init_list.manual_sequence_generator(size)

```

```

        break

    elif choice == 2:

        sequence = init_list.random_sequence_generator(size)

        break

    else:

        print("Invalid choice. Please enter 1 or 2.")

user_list = init_list.build_list_from_sequence(sequence)

max_abs = find_max_abs_element(user_list)

sum_before = sum_before_last_positive(user_list)

print_task5_results(user_list, max_abs, sum_before)
"""
Purpose: Module for list initialization (generators and builder)
Lab number: 3
Lab title: Standard data types, collections, functions, modules
Version: 1.0
Developer: Romanov Nikolai Viktorovich
Date: 08.03.2026
"""

import random

from validation import inputFloatNum

def random_sequence_generator(size):

    """
    Generator function. Yields 'size' random float numbers.
    """

```

```

    for _ in range(size):
        yield round(random.uniform(-10.0, 10.0), 2)

def manual_sequence_generator(size):
    """
    Generator function. Yields numbers typed by the user.
    """
    print(f>Please enter {size} float numbers:")
    for i in range(size):
        yield inputFloatNum(f>Element {i + 1}: ")

def build_list_from_sequence(sequence):
    """
    Takes an initialization sequence (generator) as input
    and returns a fully initialized list. (Satisfies Lab Req. 9)
    """
    return list(sequence)

```

Демонстрация задания 5:

--- Task 5: Float List Processing ---
Enter the size of the list (integer > 0): 10

Choose initialization method:
1) Manual input
2) Random generation (using generator)
Enter your choice (1 or 2): 2

=====

TASK 5 RESULTS

=====

1) Original List:
[-4.2, 0.21, -4.72, -4.83, -6.78, 9.0, -9.64, -8.35, -9.83, 9.37]

2) Element with maximum absolute value:
-9.83

3) Sum of elements before the last positive one:
-39.14

=====

Options:
1) Repeat the task
2) Exit to main menu / Quit
Enter your choice (1 or 2): █

--- Task 5: Float List Processing ---
Enter the size of the list (integer > 0): 5

Choose initialization method:
1) Manual input
2) Random generation (using generator)
Enter your choice (1 or 2): 1
Please enter 5 float numbers:
Element 1: 1
Element 2: -2
Element 3: 3
Element 4: -4
Element 5: 5

=====

TASK 5 RESULTS

=====

1) Original List:
[1.0, -2.0, 3.0, -4.0, 5.0]

2) Element with maximum absolute value:
5.0

3) Sum of elements before the last positive one:
-2.0

=====

Options:
1) Repeat the task
2) Exit to main menu / Quit
Enter your choice (1 or 2): █

Также в отдельных модулях были реализованы проверка ввода с декоратором и обработкой конкретных исключений, возможность запуска задания заново без выхода из программы и управление всеми заданиями также в отдельном модуле:

1) Проверка ввода:

```
"""
Purpose: Input validation module (decorator and input functions)
Lab number: 3
Lab title: Standard data types, collections, functions, modules
Version: 1.0
Developer: Romanov Nikolai Viktorovich
Date: 08.03.2026
"""

def userInputNum(func):
    """
    Decorator to handle user input errors.
    Repeats the input prompt until the user enters valid data.
    """
    def wrapper(prompt):
        while True:
            try:
                return func(prompt)
            except ValueError:
                print("Error: Wrong input, please try again!\n")
        return wrapper

    @userInputNum
    def inputIntNum(prompt):
```

```

"""
Requests an integer value from the user.
"""

return int(input(prompt))

@userInputNum
def inputFloatNum(prompt):
    """
    Requests a float value from the user.
    """
    return float(input(prompt))

@userInputNum
def inputStr(prompt):
    """
    Requests a non-empty string from the user.
    """
    user_str = input(prompt).strip()
    if not user_str:
        print("String cannot be empty")
        raise ValueError()
    return user_str

@userInputNum
def inputEpsilon(prompt):
    """

```

Requests an epsilon (accuracy) value greater than 0.

"""

```
eps = float(input(prompt))
```

```
if eps <= 0:
```

```
    print("Epsilon must be greater than 0.")
```

```
    raise ValueError()
```

```
return eps
```

@userInputNum

```
def inputArgumentX(prompt):
```

"""

Requests an argument X where $|X| < 1$.

"""

```
x = float(input(prompt))
```

```
if abs(x) >= 1:
```

```
    print("Absolute value of X must be less than 1.")
```

```
    raise ValueError()
```

```
return x
```

@userInputNum

```
def inputSizeList(prompt):
```

"""

Requests the size of a list (must be an integer > 0).

"""

```
size = int(input(prompt))
```

```
if size <= 0:
```

```
    print("Size must be a positive integer.")
```

```
        raise ValueError()

    return size
```

--- Task 5: Float List Processing ---

Enter the size of the list (integer > 0): 0

Size must be a positive integer.

Error: Wrong input, please try again!

Enter the size of the list (integer > 0): adsf

Error: Wrong input, please try again!

Enter the size of the list (integer > 0):

Error: Wrong input, please try again!

Enter the size of the list (integer > 0): █

2) Запуск заданий заново без выхода:

```
"""
Purpose: Menu and application loop module

Lab number: 3

Lab title: Standard data types, collections, functions, modules

Version: 1.0

Developer: Romanov Nikolai Viktorovich

Date: 08.03.2026

"""

from validation import inputIntNum

def main_loop(task_func):
    """
    Executes the passed task function and asks the user
```



```
if they want to repeat the task or exit.

"""

while True:

    task_func()

    print("\nOptions:")
    print("1) Repeat the task")
    print("2) Exit to main menu / Quit")

    while True:

        choice = inputIntNum("Enter your choice (1 or 2): ")

        if choice == 1:

            print("\n" + "="*50 + "\n")

            break

        elif choice == 2:

            return

        else:

            print("Incorrect choice, please try again.")
```

```

--- Task 2: Count positive numbers ---
Enter integers one by one.
To finish the input and calculate the result, enter the number 10.

Enter an integer: 10
-----
Total count of positive numbers entered: 0
-----

Options:
1) Repeat the task
2) Exit to main menu / Quit
Enter your choice (1 or 2): 1

=====

--- Task 2: Count positive numbers ---
Enter integers one by one.
To finish the input and calculate the result, enter the number 10.

```

3) Управлениями всеми заданиями из другого модуля, тестирование:

```

"""
Purpose: Main application entry point and Main Menu
Lab number: 3
Lab title: Standard data types, collections, functions, modules
Version: 1.0
Developer: Romanov Nikolai Viktorovich
Date: 08.03.2026
"""

from validation import inputIntNum
from menu import main_loop
from task1 import task1
from task2 import task2
from task3 import task3

```

```

from task4 import task4

from task5 import task5


def main():
    """
    Main entry point. Displays the Main Menu and routes the user
    to the selected task.
    """

    while True:

        print("\n" + "="*40)

        print(f"{'MAIN MENU':^40}")

        print("="*40)

        print("1) Run Task 1")
        print("2) Run Task 2")
        print("3) Run Task 3")
        print("4) Run Task 4")
        print("5) Run Task 5")
        print("0) Exit Program")

        print("="*40)

        choice = inputIntNum("Enter task number (0-5): ")

        match choice:
            case 1:
                main_loop(task1)
            case 2:
                main_loop(task2)
            case 3:

```

```
        main_loop(task3)

    case 4:

        main_loop(task4)

    case 5:

        main_loop(task5)

    case 0:

        break

    case _:

        print("Invalid choice. Please select an existing
task.")

if __name__ == "__main__":
    main()
```

```
=====
                        MAIN MENU
=====
1) Run Task 1
2) Run Task 2
3) Run Task 3
4) Run Task 4
5) Run Task 5
0) Exit Program
=====
Enter task number (0-5): 4

--- Task 4: Text Analysis ---
```